Study Material - Generated by AI

CS3301 - Data Structures

Topic: Stack

1. SUMMARY

A stack is a fundamental data structure that follows the Last-In-First-Out (LIFO) principle. It is a linear data structure that supports insertion and deletion operations from a fixed position, known as the top of the stack. The reference material discusses the implementation of a stack using an array and a linked list, and mentions its applications in expression conversion and evaluation. The stack ADT provides basic operations such as push, pop, and peek.

2. SHORT NOTES

Stack ADT is a linear data structure that follows the Last-In-First-Out (LIFO) principle. It consists of a sequence of elements, where the topmost element can be accessed or removed. The stack ADT provides the following operations:

- **Push**: Adds an element to the top of the stack.
- **Pop**: Removes the top element from the stack.
- **Peek**: Returns the top element of the stack without removing it.

The stack data structure can be implemented using an array or a linked list. Array implementation of stack involves using a fixed-size array to store the elements, while linked list implementation uses a dynamic memory allocation.

Application of stack data structure:

- Expression conversion and evaluation: Stack is used to evaluate postfix expressions, where an operator is pushed onto the stack when it is encountered in the expression, and the operands are popped from the stack when the operator is encountered again.

Types of stacks:

- **Simple Stack**: A basic stack implementation where elements are inserted and deleted from the top.
- **Circular Stack**: A stack implementation where the top element points to the bottom element, and the last element points to the top element.
- **Priority Stack**: A stack implementation where elements are inserted based on their priority.

3. IMPORTANT QUESTIONS

1. What is the Last-In-First-Out (LIFO) principle of a stack data structure?
2. What are the basic operations provided by a stack ADT?
3. How is a stack implemented using an array and a linked list?
4. What is the application of stack data structure in expression conversion and evaluation?
5. What is the difference between a simple stack and a circular stack?
6. How is a priority stack implementation different from a simple stack?
7. What are the advantages of using a stack data structure?
8. How can a stack be used to evaluate postfix expressions?
9. What is the significance of the "top" element in a stack data structure?
10. Can a stack be implemented using a queue data structure? Why or why not?

4. MCQs

Q1. What is the Last-In-First-Out (LIFO) principle of a stack data structure?

a) First-In-First-Out

b) Last-In-First-Out

c) Random Access

d) Sorted Access

Answer: b) Last-In-First-Out

Q2. Which of the following is a basic operation provided by a stack ADT?

a) Push

b) Pop

c) Peek

d) All of the above

Answer: d) All of the above

Q3. How is a stack implemented using a linked list?

a) Each element points to its previous element

b) Each element points to its next element

c) Elements are stored in a contiguous block of memory

d) Elements are stored in a dynamically allocated memory

Answer: b) Each element points to its next element

Q4. What is the application of stack data structure in expression conversion and evaluation?

a) Prefix expression evaluation

b) Postfix expression evaluation

c) Infix expression evaluation

d) Neither of the above

Answer: b) Postfix expression evaluation

Q5. What is the difference between a simple stack and a circular stack?

a) A simple stack has a fixed size, while a circular stack has a variable size.

b) A simple stack has a variable size, while a circular stack has a fixed size.

c) A simple stack uses a linked list, while a circular stack uses an array.

d) A simple stack uses an array, while a circular stack uses a linked list.

Answer: a) A simple stack has a fixed size, while a circular stack has a variable size.

Q6. How is a priority stack implementation different from a simple stack?

a) A priority stack has a lower priority than a simple stack.

b) A priority stack has a higher priority than a simple stack.

c) A priority stack is used for inserting elements with different priorities.

d) A priority stack is used for deleting elements with different priorities.

Answer: c) A priority stack is used for inserting elements with different priorities.

Q7. What are the advantages of using a stack data structure?

a) Faster insertion and deletion operation

b) Efficient memory usage

c) Easy implementation using an array or linked list

d) All of the above

Answer: d) All of the above

Q8. Can a stack be implemented using a queue data structure? Why or why not?

a) Yes, because a queue and a stack both follow the FIFO principle.

b) No, because a queue and a stack follow different principles.

c) Yes, because a queue can be used to implement a stack using a modified insertion and deletion operation.

d) No, because a queue is a linear data structure, while a stack is a non-linear data structure.

Answer: b) No, because a queue and a stack follow different principles.

Q9. What is the significance of the "top" element in a stack data structure?

a) The top element is the first element inserted into the stack.

b) The top element is the last element inserted into the stack.

c) The top element is the only element that can be accessed or removed.

d) The top element is the only element that cannot be accessed or removed.

Answer: b) The top element is the last element inserted into the stack.

Q10. Which of the following is a correct statement about a stack data structure?

a) A stack is a linear data structure that follows the LIFO principle.

b) A stack is a non-linear data structure that follows the FIFO principle.

c) A stack is a linear data structure that follows the FIFO principle.

d) None of the above

Answer: a) A stack is a linear data structure that follows the LIFO principle.